

 INSTITUTO FEDERAL CEARÁ Campus Canindé	Curso: Análise e Desenvolvimento de Sistemas	Data: 07/08/2026
	Disciplina: Programação Web II	4º semestre
	Professora: Maria Luciana Almeida Pereira	Turno: Noite

Lista de exercícios

1. Faça um programa que leia 3 números, some-os e exiba a média entre eles.
2. Faça um programa que leia do usuário uma temperatura em Fahrenheit, converta-a para graus Celsius e escreva o novo valor na tela. A fórmula de conversão de Fahrenheit (F) para Celsius é $C = ((F - 32) * 5) / 9$. Exemplo: 100 Fahrenheit = 37,77 Celsius.
3. Faça um programa que leia 2 números, some-os e exiba uma mensagem com o resultado apenas se o resultado for maior que 100.
4. Crie uma função que receba um valor e uma porcentagem como parâmetros. A função deve retornar o valor acrescido da porcentagem indicada.
5. Neste exercício, você escreverá um código para preparar uma lasanha requintada de seu livro de receitas favorito. Você tem quatro tarefas relacionadas ao tempo gasto no preparo da lasanha.

1. Defina o tempo de forno esperado em minutos.

Defina a constante **EXPECTED_MINUTES_IN_OVEN** que representa quantos minutos a lasanha deve ficar no forno. De acordo com o livro de receitas, o tempo de forno esperado em minutos é 40.

2. Calcule o tempo restante do forno em minutos.

Implemente a função que toma como parâmetro **remainingMinutesInOven** os minutos reais que a lasanha ficou no forno e retorna quantos minutos a lasanha ainda tem que permanecer no forno, com base no **tempo de forno esperado em minutos** da tarefa anterior.

```
1 remainingMinutesInOven(30);
2 // => 10
```

3. Calcule o tempo de preparação em minutos

Implemente a função **preparationTimeInMinutes** que recebe o número de camadas que você adicionou à lasanha como parâmetro e retorna quantos minutos você gastou preparando a lasanha, supondo que cada camada leve 2 minutos para ser preparada.

```
1 preparationTimeInMinutes(2);
2 // => 4
```

4. Calcule o tempo total de trabalho em minutos

Implemente a função **totalTimeInMinutes** que recebe dois parâmetros: o parâmetro **numberOfLayers** é o número de camadas que você adicionou à lasanha e o parâmetro **actualMinutesInOven** é o número de

minutos que a lasanha ficou no forno. A função deve retornar quantos minutos no total você trabalhou no cozimento da lasanha, que é a soma do tempo de preparo em minutos e do tempo em minutos que a lasanha ficou no forno até o momento.

```
1 totalTimeInMinutes(3, 20);  
2 // => 26
```

6. Desenvolva uma função que receba um número como parâmetro e verifique se ele é par ou ímpar. Retorne true se for par e false se for ímpar.
7. Neste exercício, você implementará a lógica de busca para um novo RPG que um amigo está desenvolvendo. A personagem principal do jogo é Annalyn, uma garota corajosa com um cachorro de estimação feroz e leal. Infelizmente, um desastre acontece: sua melhor amiga foi sequestrada enquanto procurava frutas na floresta. Annalyn tentará encontrar e libertar sua melhor amiga, levando seu cachorro com ela nessa busca.

Depois de algum tempo seguindo o rastro de sua melhor amiga, ela encontra o acampamento onde ela está aprisionada. Descobriu-se que havia dois sequestradores: um poderoso **cavaleiro** e um **arqueiro** astuto.

Após encontrar os sequestradores, Annalyn considera quais das seguintes ações ela pode realizar:

- a. **Ataque rápido:** um ataque rápido pode ser feito se o cavaleiro estiver dormindo, pois leva tempo para ele vestir sua armadura, então ele ficará vulnerável.
- b. **Espião:** o grupo pode ser espionado se pelo menos um deles estiver acordado. Caso contrário, espionar é perda de tempo.
- c. **Sinalizar prisioneiro:** o prisioneiro pode ser sinalizado usando sons de pássaros se estiver acordado e o arqueiro estiver dormindo, pois os arqueiros são treinados em sinalização de pássaros para que possam interceptar a mensagem.
- d. **Libertar o prisioneiro:** Annalyn pode tentar entrar furtivamente no acampamento para libertar o prisioneiro. Isso é arriscado e só pode dar certo de duas maneiras:
 - i. Se Annalyn estiver com seu cachorro de estimação, ela poderá resgatar o prisioneiro se o arqueiro estiver dormindo. O cavaleiro está com medo do cachorro e o arqueiro não terá tempo de se preparar antes que Annalyn e o prisioneiro consigam escapar.
 - ii. Se Annalyn não estiver com seu cachorro, ela e o prisioneiro devem ser muito sorrateiros! Annalyn pode libertar o prisioneiro se ele estiver acordado e o cavaleiro e o arqueiro estiverem dormindo, mas se o prisioneiro estiver dormindo, eles não poderão ser resgatados: o prisioneiro se assustaria com a aparição repentina de Annalyn e acordaria o cavaleiro e o arqueiro.

Você tem **quatro tarefas**: implementar a lógica para determinar se as ações acima estão disponíveis com base no estado dos três personagens encontrados na floresta e se o cachorro de estimação de Annalyn está presente ou não.

1. Verifique se a ação “Ataque Rápido” é possível

Implemente uma função chamada **canExecuteFastAttack** que recebe um valor booleano que indica se o cavaleiro está acordado. Esta função retorna **true** se a ação “Ataque Rápido” está disponível com base no estado do personagem. Caso contrário, retorna **false**:

```
1  const knightIsAwake = true;
2  canExecuteFastAttack(knightIsAwake);
3  // => false
```

2. Verifique se a ação “Espiar” é possível

Implemente uma função chamada **canSpy** que recebe três valores booleanos, indicando se o cavaleiro, o arqueiro e o prisioneiro, respectivamente, estão acordados. A função retorna **true** se a ação “Espiar” está disponível com base no estado dos personagens. Caso contrário, retorna **false**:

```
1  const knightIsAwake = false;
2  const archerIsAwake = true;
3  const prisonerIsAwake = false;
4  canSpy(knightIsAwake, archerIsAwake, prisonerIsAwake);
5  // => true
```

3. Verifique se a ação “Sinalizar Prisioneiro” é possível

Implemente uma função chamada **canSignalPrisoner** que recebe dois valores booleanos, indicando se o arqueiro e o prisioneiro, respectivamente, estão acordados. A função retorna **true** se a ação “Sinalizar Prisioneiro” está disponível com base no estado dos personagens. Caso contrário, retorna **false**:

```
1  const archerIsAwake = false;
2  const prisonerIsAwake = true;
3  canSignalPrisoner(archerIsAwake, prisonerIsAwake);
4  // => true
```

4. Verifique se a ação “Libertar Prisioneiro” é possível

Implemente uma função chamada **canFreePrisoner** que recebe quatro valores booleanos. Os três primeiros parâmetros indicam se o cavaleiro, o arqueiro e o prisioneiro, respectivamente, estão acordados. O último parâmetro indica se o cachorro de estimação de Annalyn está presente. A função retorna **true** se a ação “Libertar Prisioneiro” está disponível com base no estado dos personagens e na presença do cachorro de estimação de Annalyn. Caso contrário, retorna **false**:

```
1  const knightIsAwake = false;
2  const archerIsAwake = true;
3  const prisonerIsAwake = false;
4  const petDogIsPresent = false;
5  canFreePrisoner(knightIsAwake, archerIsAwake, prisonerIsAwake, petDogIsPresent);
6  // => false
```

8. Para fazer um bolo você decidiu separar os ingredientes em duas listas de ingredientes:

```
let ingredientesBase = ["farinha", "ovos", "leite"];
let ingredientesExtras = ["chocolate", "morango"];
```

Depois, você decidiu criar uma lista chamada **ingredientes** que contém os ingredientes extras e base, respectivamente.

Você viu em um artigo que se adicionar "baunilha" no bolo ele fica mais fofinho, então você adicionou este ingrediente ao final da lista.

Mas, você descobriu que isto é um mito, a baunilha é um aromatizante e é usada para dar sabor e aroma ao bolo, e portanto não tem nenhuma relação com a textura do bolo. Você então, removeu o último ingrediente que foi adicionado.

Depois inverteu a ordem dos ingredientes restantes e gerou uma string com todos os ingredientes separados por ">" que foi impresso na tela.

Utilize os métodos de array para executar cada uma dessas tarefas.